

Pruebas de Regresión — Módulo Bolsillos (FuBanking)

Asignatura	Validación y Verificación de Software
Proyecto	FuBanking — Banco digital
Módulo bajo prueba	Bolsillos (<i>Pockets</i>) + Depósito (módulo Cuentas)
Fase	Pruebas de regresión + aserciones fluidas (<i>fluent assertions</i>) + integración continua
Rama	<code>tests/bolsillos-regresion-fluent</code> (sobre <code>devops/jenkins-gitops</code>)
Fecha	2026-09-29

1. Objetivo

1. Construir una **suite de regresión automatizada** para **seis funcionalidades** —las cinco del módulo Bolsillos (**crear, consultar, actualizar, eliminar y transferir**) más **depositar dinero** (módulo Cuentas)— que fije el comportamiento observable de la API y detecte cualquier cambio no intencional.
2. Escribir las pruebas con **aserciones fluidas**, convirtiendo también las pruebas unitarias existentes del módulo.
3. **Demostrar** con la suite que los cambios de código (refactor de `UpdatePocket`, correcciones de SonarQube del equipo) **no introdujeron regresiones**, ejecutándola antes y después.
4. Dejar la suite dentro del **pipeline de Jenkins** del equipo (build → pruebas → SonarQube → Docker).

2. Alcance

2.1 Funcionalidades y endpoints

Todas las rutas exigen `Authorization: Bearer <JWT>`.

#	Funcionalidad	Método y ruta	Validador	Caso de uso
1	Crear bolsillo	<code>POST /api/v1/pockets</code>	<code>createPocketSchema</code>	<code>CreatePocket</code>
2	Consultar bolsillos	<code>GET /api/v1/pockets/account/:accountId</code>	—	<code>GetAccountPockets</code>
3	Actualizar bolsillo	<code>PATCH /api/v1/pockets/:pocketId</code>	<code>updatePocketSchema</code>	<code>UpdatePocket</code>
4	Eliminar bolsillo	<code>DELETE /api/v1/pockets/:pocketId</code>	—	<code>DeletePocket</code>
5	Transferir entre bolsillos	<code>POST /api/v1/pockets/transfer</code>	<code>transferPocketSchema</code>	<code>TransferPocketBalance</code>
6	Depositar dinero	<code>POST /api/v1/accounts/:id/deposit</code>	— (lo valida el caso de uso)	<code>DepositMoney</code>

2.2 ¿Por qué el depósito como sexta funcionalidad?

- **No la trabaja nadie más del equipo.** Revisando autores, ramas y pruebas del repositorio: Créditos (Andrés), Autenticación y Perfil (Tomás), Tarjetas virtuales (Dubin) y Bolsillos (yo). El módulo Cuentas no tenía pruebas de nadie.
- **Está ligada a Bolsillos:** el depósito aumenta el **saldo disponible**, que es justo lo que los bolsillos apartan. La suite lo prueba con un flujo cruzado (RG-DE-15).
- **Misma estructura** que los casos de uso de Bolsillos (validar monto → cuenta existe → es del usuario → está activa → actualizar saldo → notificar), así que reutiliza los mismos fakes y el mismo escenario.

2.3 Fuera de alcance

- **Frontend** (la regresión se hizo a nivel de API del backend).
- **Supabase real.** La persistencia se sustituye por repositorios en memoria; el defecto **D-07** (respaldo en memoria de `SupabasePocketRepository`) no es observable a este nivel.
- **Los archivos `*.routes.ts`:** instancian los repositorios de Supabase al importarse, así que la app de prueba replica su cableado (mismos verbos, rutas y middleware), igual que hace `createTestApp` del equipo con las rutas de autenticación.

3. Estrategia

3.1 Nivel: API con supertest (patrón del equipo)

Se sigue el patrón del equipo en `tests/integration/*.integration.test.ts`: una función arma una app Express con dependencias en memoria y **supertest** hace peticiones HTTP reales contra ella. Para este alcance se creó `createPocketTestApp`, que monta las rutas de Bolsillos y la de depósito sobre **los mismos repositorios**.

```
supertest → authMiddleware (JWT real) → validador zod → PocketController / AccountController  
→ caso de uso → repositorios en memoria → errorHandler → respuesta JSON
```

Componente	¿Real o doble?
<code>authMiddleware + JwtTokenService</code> (firma y verificación de tokens)	Real
Validadores zod (<code>pocket.validators.ts</code>)	Real
<code>PocketController</code> , <code>AccountController</code> , <code>sendSuccess</code> / <code>sendError</code>	Real
Casos de uso (<code>CreatePocket</code> , ..., <code>DepositMoney</code>) y entidades de dominio	Real
<code>errorHandler</code>	Real
Repositorios de cuentas, bolsillos y notificaciones	Fake en memoria

3.2 Repositorios en memoria con semántica de copia

Los fakes nuevos (`tests/fakes/InMemory{Account,Pocket,Notification}Repository.ts`) guardan filas planas y reconstruyen la entidad en cada lectura, como lo haría una base de datos. Si

un caso de uso modificara una entidad y **olvidara persistirla**, un fake que devuelve la misma instancia no lo notaría; con copias, el cambio no queda guardado y la prueba falla.

3.3 Escenario base

```
TITULAR — CUENTA (saldo disponible 1.000.000, ACTIVA, BA1111111101)
          └─ VIAJE      200.000
             └─ EMERGENCIAS  50.000
          └─ CUENTA_SIN_BOLSILLOS (saldo 300.000)
INTRUSO  — CUENTA_AJENA (saldo 500.000)
          └─ AJENO      30.000
```

Cada prueba monta el escenario desde cero (`beforeEach`), así que las pruebas son independientes.

3.4 Qué verifica cada caso

Aspecto	Cómo se verifica
Código HTTP	<code>expect(res.status).to.equal(201)</code>
Contrato de respuesta	claves exactas del bolsillo / cuenta pública, UUID, fechas ISO, <code>message</code>
Contrato de errores	código y mensaje exactos (<code>ERR.FORBIDDEN</code> , <code>validationError(...)</code>), porque el frontend muestra el mensaje del backend
Estado persistido	saldo y bolsillos leídos del repositorio después de la operación
Invariante de dinero	<code>saldo disponible + Σ bolsillos</code> se conserva al crear, actualizar, eliminar y transferir; el depósito lo aumenta exactamente en el monto
Efectos secundarios	notificación emitida (título, tipo, usuario, mensaje)
Sin efectos parciales	tras un error (401, 403, 400) el estado queda idéntico al previo
Valores límite	exactamente el disponible pasa; un peso más se rechaza

3.5 Defectos conocidos: `it.fails`

Los defectos documentados (**D-01 ... D-05** de rondas anteriores y **D-08**, encontrado ahora en el depósito) no se corrigen en esta entrega. Cada uno tiene una prueba que afirma la **regla correcta** marcada con `it.fails` y su ID: la suite queda en verde mientras el defecto exista y **se pone en rojo en cuanto alguien lo corrija**.

4. Aserciones fluidas (Chai BDD)

4.1 Elección

Se usó el **estilo BDD de Chai**, que Vitest trae integrado. *FluentAssertions* es una librería de .NET; en JavaScript/TypeScript su equivalente estándar es Chai BDD. No requiere dependencias nuevas y permite **encadenar** condiciones que se leen como una frase:

```
expect(res.body.data)
  .to.have.all.keys('id', 'accountId', 'name', 'amount', 'createdAt', 'updatedAt')
  .and.to.include({ accountId: CUENTA, name: 'Carro', amount: 120_000 });
```

4.2 Equivalencias aplicadas

Estilo Jest (antes)	Estilo fluido Chai BDD (ahora)
<code>expect(x).toBe(v)</code>	<code>expect(x).to.equal(v)</code>
<code>expect(o.a).toBe(1); expect(o.b).toBe(2)</code>	<code>expect(o).to.include({ a: 1, b: 2 })</code>
<code>expect(account?.balance).toBe(800_000)</code>	<code>expect(account).to.have.property('balance', 800_000)</code>
<code>expect(x).toEqual({...})</code>	<code>expect(x).to.deep.equal({...})</code>
<code>expect(list).toHaveLength(2)</code>	<code>expect(list).to.be.an('array').with.lengthOf(2)</code>
<code>expect(list).toEqual([])</code>	<code>expect(list).to.be.an('array').that.is.empty</code>
<code>expect(x).toBeNull()</code>	<code>expect(x).to.be.null</code>
<code>expect(d).toBeInstanceOf(Date)</code>	<code>expect(d).to.be.an.instanceOf(Date)</code>
<code>expect(s).toContain('Regalo')</code>	<code>expect(o).to.have.property('message').that.includes('Regalo')</code>
<code>expect(fn).toThrowError(/re/)</code>	<code>expect(fn).to.throw(AppError, /re/).with.property('code', '...')</code>
<code>await expect(p).rejects.toMatchObject({...})</code>	<code>await expect(p).rejects.to.include({...})</code>
<code>expect(mock).toHaveBeenCalledWith(a, b)</code>	<code>expect(mock).to.have.been.calledWith(a, b)</code>
<code>expect(mock).toHaveBeenCalledTimes(1)</code>	<code>expect(mock).to.have.been.calledOnce</code>

4.3 Ejemplo antes / después

```
// Antes (CreatePocket.test.ts)
expect(spyNotif.saveCallCount).toBe(1);
expect(spyNotif.savedNotifications[0].title).toBe('Bolsillo creado');
expect(spyNotif.savedNotifications[0].message).toContain('Regalo');

// Ahora
expect(spyNotif.saveCallCount).to.equal(1);
expect(spyNotif.savedNotifications[0])
  .to.include({ title: 'Bolsillo creado' })
  .and.to.have.property('message').that.includes('Regalo');
```

4.4 Alcance

Suite	Pruebas	Estado
<code>unit/pocket/</code> — <code>CreatePocket</code> , <code>UpdatePocket</code> , <code>DeletePocket</code> , <code>TransferPocketBalance</code> , <code>GetAccountPockets</code> , <code>Pocket.entity</code>	58	Convertidas a Chai BDD
<code>unit/account/DepositMoney.test.ts</code> (funcionalidad 6, con los 5 dobles de prueba)	12	Nueva, en Chai BDD
<code>regression/pocket/</code> + <code>regression/account/</code>	94	Nueva, en Chai BDD

- Se preservó lo que verifica cada prueba convertida. Solo en `Pocket.entity.test.ts` las aserciones de excepción se volvieron más precisas: ahora verifican también el tipo (`AppError`) y el código de error.
- Se completaron los dobles de `test-doubles.ts` (Fake, Stub y Mock de cuentas) con `updateStatus`, que `IAccountRepository` exige y que causaba errores de tipos aunque Vitest los ignorara.

5. Catálogo de casos de regresión

94 casos: 89 reglas vigentes + 5 defectos conocidos (*it.fails*).

5.1 Crear — `regression/pocket/crear.regression.test.ts` (21)

ID	Caso	Esperado
RG-CR-01	Contrato público del bolsillo creado	201, claves exactas, UUID, fecha ISO
RG-CR-02	Persiste y descuenta del saldo disponible	saldo 880.000, total de la cuenta intacto
RG-CR-03	Notifica al titular	1 notificación "Bolsillo creado" con el nombre
RG-CR-04	Recorta espacios del nombre	" Casa " → "Casa"
RG-CR-05	Monto como texto numérico	"150000" → 150000 (número)
RG-CR-06	Monto cero	201, saldo sin cambios
RG-CR-07	Límite: todo el saldo disponible	201, saldo 0
RG-CR-08	Límite: nombre de 150 caracteres	201
RG-CR-09	<code>accountId</code> no UUID	400 <code>VALIDATION_ERROR</code> (campo <code>accountId</code>)
RG-CR-10	Monto negativo	400 <code>VALIDATION_ERROR</code> (campo <code>amount</code>)
RG-CR-11	Nombre vacío	400 <code>VALIDATION_ERROR</code> (campo <code>name</code>)
RG-CR-12	Nombre de 151 caracteres	400 <code>VALIDATION_ERROR</code> (campo <code>name</code>)
RG-CR-13	Sin token	401 <code>UNAUTHORIZED</code>
RG-CR-14	Token inválido	401 <code>TOKEN_INVALID</code>
RG-CR-15	Cuenta ajena	403 <code>FORBIDDEN</code> , cuenta ajena intacta
RG-CR-16	Cuenta inexistente	404 <code>ACCOUNT_NOT_FOUND</code>
RG-CR-17	Cuenta bloqueada	400 <code>ACCOUNT_NOT_OPERATIONAL</code> , estado intacto
RG-CR-18	Supera el disponible por \$1	400 <code>INSUFFICIENT_AVAILABLE_BALANCE</code> , sin notificación
RG-CR-D02	[D-02] Monto booleano	Esperado 400 · hoy 201 con bolsillo de \$1
RG-CR-D03	[D-01/D-03] Nombre de solo espacios	Esperado 400 · hoy 201 con nombre vacío
RG-CR-D05	[D-05] Bolsillo que cabe en el disponible	Esperado 201 · hoy 400

5.2 Consultar — `regression/pocket/consultar.regression.test.ts` (11)

ID	Caso	Esperado
RG-CO-01	Lista en forma pública	200, 2 bolsillos con claves exactas y fechas ISO
RG-CO-02	Nombre y monto persistidos	Viaje 200.000 · Emergencias 50.000
RG-CO-03	No mezcla otras cuentas	ningún bolsillo de <code>CUENTA_AJENA</code>
RG-CO-04	Cuenta sin bolsillos	200, lista vacía
RG-CO-05	Lectura después de escritura	el bolsillo recién creado aparece idéntico
RG-CO-06	Cuenta bloqueada	200 (consultar no exige cuenta operativa)
RG-CO-07	Es de solo lectura	estado y notificaciones sin cambios
RG-CO-08	Sin token	401 <code>UNAUTHORIZED</code>
RG-CO-09	Cuenta ajena	403 <code>FORBIDDEN</code>
RG-CO-10	Cuenta inexistente	404 <code>ACCOUNT_NOT_FOUND</code>
RG-CO-11	Id de cuenta mal formado	404 <code>ACCOUNT_NOT_FOUND</code> (la ruta no valida formato)

5.3 Actualizar — `regression/pocket/actualizar.regression.test.ts` (19)

ID	Caso	Esperado
RG-AC-01	Renombrar	200, contrato público, saldo sin cambios
RG-AC-02	Persistencia y marcas de tiempo	nombre guardado, <code>createdAt</code> igual, <code>updatedAt</code> nuevo
RG-AC-03	Aumentar monto	saldo -100.000, total intacto
RG-AC-04	Reducir monto	saldo +160.000, total intacto
RG-AC-05	Nombre y monto a la vez	ambos persistidos
RG-AC-06	Mismo monto	saldo sin cambios
RG-AC-07	Límite: disponible + monto actual	200, saldo 0
RG-AC-08	Recorta espacios del nombre	" Playa " → "Playa"
RG-AC-09	Notifica al titular	1 notificación "Bolsillo actualizado"
RG-AC-10	Cuerpo vacío	400 <code>VALIDATION_ERROR</code> (campo <code>general</code>)
RG-AC-11	Monto negativo	400 <code>VALIDATION_ERROR</code> (campo <code>amount</code>)
RG-AC-12	Nombre vacío	400 <code>VALIDATION_ERROR</code> (campo <code>name</code>)
RG-AC-13	Nombre de solo espacios	400 <code>INVALID_POCKET_NAME</code> (lo ataja el caso de uso)
RG-AC-14	Supera el límite por \$1	400 <code>INSUFFICIENT_AVAILABLE_BALANCE</code> , nada modificado
RG-AC-15	Bolsillo inexistente	404 <code>POCKET_NOT_FOUND</code>
RG-AC-16	Bolsillo ajeno	403 <code>FORBIDDEN</code> , intacto
RG-AC-17	Cuenta bloqueada	400 <code>ACCOUNT_NOT_OPERATIONAL</code> , intacto
RG-AC-18	Sin token	401 <code>UNAUTHORIZED</code>
RG-AC-D04	[D-04] Monto como texto numérico	Esperado 200 (como al crear) · hoy 400

5.4 Eliminar — `regression/pocket/eliminar.regression.test.ts` (10)

ID	Caso	Esperado
RG-EL-01	Foto pública del eliminado	200 con el bolsillo tal como estaba
RG-EL-02	Borra y devuelve el monto	saldo +200.000, total intacto
RG-EL-03	Notifica al titular	1 notificación "Bolsillo eliminado"
RG-EL-04	Bolsillo en cero	saldo sin cambios
RG-EL-05	Desaparece de la consulta	la lista ya no lo incluye
RG-EL-06	Segundo DELETE	404, el dinero no se devuelve dos veces
RG-EL-07	Bolsillo inexistente	404 <code>POCKET_NOT_FOUND</code>
RG-EL-08	Bolsillo ajeno	403 <code>FORBIDDEN</code> , no se elimina
RG-EL-09	Cuenta bloqueada	400 <code>ACCOUNT_NOT_OPERATIONAL</code> , no se elimina
RG-EL-10	Sin token	401 <code>UNAUTHORIZED</code> , el bolsillo sigue

5.5 Transferir — `regression/pocket/transferir.regression.test.ts` (15)

ID	Caso	Esperado
RG-TR-01	Ambos bolsillos actualizados	200, origen 100.000, destino 150.000
RG-TR-02	Persistencia	ambos guardados, saldo de la cuenta intacto
RG-TR-03	Notifica el movimiento	mensaje con origen y destino
RG-TR-04	Límite: todo el origen	origen 0, destino 250.000
RG-TR-05	Ida y vuelta	total de la cuenta intacto
RG-TR-06	Monto cero	400 <code>VALIDATION_ERROR</code> (campo <code>amount</code>)
RG-TR-07	Ids no UUID	400 <code>VALIDATION_ERROR</code> (<code>fromPocketId</code> , <code>toPocketId</code>)

ID	Caso	Esperado
RG-TR-08	Hacia sí mismo	400 <code>INVALID_TRANSFER_TARGET</code>
RG-TR-09	Origen inexistente	404 <code>SOURCE_POCKET_NOT_FOUND</code>
RG-TR-10	Destino inexistente	404 <code>TARGET_POCKET_NOT_FOUND</code>
RG-TR-11	Cuentas distintas	400 <code>POCKETS_DIFFERENT_ACCOUNT</code> , no mueve dinero
RG-TR-12	Un tercero lo intenta	403 <code>FORBIDDEN</code> , no mueve dinero
RG-TR-13	Cuenta bloqueada	400 <code>ACCOUNT_NOT_OPERATIONAL</code> , no mueve dinero
RG-TR-14	Origen insuficiente por \$1	400 <code>INSUFFICIENT_POCKET_BALANCE</code> , sin notificación
RG-TR-15	Sin token	401 <code>UNAUTHORIZED</code>

5.6 Depositar — `regression/account/depositar.regression.test.ts` (16)

ID	Caso	Esperado
RG-DE-01	Contrato de la cuenta pública	200, claves exactas, saldo +100.000, estado ACTIVA
RG-DE-02	Persiste el saldo sin tocar bolsillos	saldo 1.100.000, bolsillos intactos
RG-DE-03	Notifica al titular	"Depósito realizado", tipo SISTEMA, <code>****1101</code>
RG-DE-04	Depósitos sucesivos	se acumulan (+30.000), 3 notificaciones
RG-DE-05	Monto como texto numérico	<code>"50000"</code> → +50.000
RG-DE-06	Monto con decimales	+1.500,5
RG-DE-07	Monto cero	400 <code>INVALID_AMOUNT</code> , nada cambia
RG-DE-08	Monto negativo	400 <code>INVALID_AMOUNT</code> , nada cambia
RG-DE-09	Monto no numérico	400 <code>INVALID_AMOUNT</code> , nada cambia
RG-DE-10	Sin monto	400 <code>INVALID_AMOUNT</code> , nada cambia
RG-DE-11	Sin token	401 <code>UNAUTHORIZED</code>
RG-DE-12	Cuenta ajena	403 <code>FORBIDDEN</code> , intacta
RG-DE-13	Cuenta inexistente	404 <code>ACCOUNT_NOT_FOUND</code>
RG-DE-14	Cuenta bloqueada	400 <code>ACCOUNT_INACTIVE</code> , intacta
RG-DE-15	Integración con Bolsillos	un bolsillo de 400.000 se rechaza; tras depositar 100.000, se crea
RG-DE-D08	[D-08] Monto booleano	<i>Esperado</i> 400 · hoy 200 y deposita \$1

5.7 Ciclo de vida — `regression/pocket/ciclo-de-vida.regression.test.ts` (2)

ID	Caso	Esperado
RG-FL-01	Crear → consultar → actualizar → transferir → eliminar	total 300.000 conservado en cada paso; 5 notificaciones en orden
RG-FL-02	Aislamiento entre usuarios	4 intentos cruzados → 403; nada cambia

6. Defectos y hallazgos

6.1 Defectos cubiertos por la suite (`it.fails`)

Defecto	Descripción	Prueba	Regla que afirma	Comportamiento actual
---------	-------------	--------	------------------	-----------------------

Defecto	Descripción	Prueba	Regla que afirma	Comportamiento actual
D-01 / D-03	Nombre de solo espacios pasa el validador y la entidad no valida el nombre al construir	RG-CR-D03	400	201, bolsillo con nombre ""
D-02	<code>z.coerce.number()</code> convierte <code>true</code> en <code>1</code> al crear un bolsillo	RG-CR-D02	400	201, bolsillo de \$1
D-04	<code>amount</code> se trata distinto al crear (<code>coerce</code>) y al actualizar (<code>number</code>)	RG-AC-D04	200	400 <code>VALIDATION_ERROR</code>
D-05	Al crear un bolsillo, lo apartado se cuenta dos veces	RG-CR-D05	201	400 <code>INSUFFICIENT_AVAILABLE_BALANCE</code>
D-08	El controlador del depósito hace <code>Number(amount): true</code> se convierte en <code>1</code>	RG-DE-D08	400	200, deposita \$1

Cada `it.fails` se ejecutó como prueba normal para confirmar que falla **justo en la aserción del defecto** (`expected 201 to equal 400`, `expected 400 to equal 201`, `expected 400 to equal 200`, `expected 200 to equal 400`).

6.2 Otros hallazgos (documentados, no corregidos)

- **Cableado del depósito en main.** En `main, account.routes.ts` construye `new DepositMoney(accountRepository, transactionRepository, notificationRepository)`, pero el caso de uso solo recibe dos dependencias: el **repositorio de transacciones queda como repositorio de notificaciones**. La rama del pipeline ya lo corrigió (commit `d3ba8e9`), y por eso este trabajo se integra sobre ella.
- **Códigos de error inconsistentes entre módulos.** Para el mismo problema, Bolsillos responde `ACCOUNT_NOT_FOUND: "Cuenta no encontrada"` y `ACCOUNT_NOT_OPERATIONAL`, mientras que Cuentas responde `"La cuenta no existe"` y `ACCOUNT_INACTIVE`. La suite fija ambos contratos tal como están.
- **La descripción del depósito se ignora.** `DepositMoneyDto.description` se recibe pero nunca se usa, y el depósito no genera registro de transacción.
- **Prueba unitaria que fija D-05.** *"lanza `INSUFFICIENT_AVAILABLE_BALANCE` cuando no hay saldo disponible"* (`CreatePocket.test.ts`) espera el rechazo de un bolsillo que, con la regla correcta, sí cabe. Debe ajustarse cuando se corrija D-05.

7. Evidencia: la suite verifica el funcionamiento después de un cambio

7.1 Versiones comparadas

	Versión	Descripción
Antes	6102a28	Código previo a cualquier refactor (pruebas unitarias AAA + 5 dobles)
Después	devops/jenkins-gitops (3205dc1)	Código actual del pipeline: refactor de Bolsillos + correcciones de SonarQube del equipo

Cambios de código de producción entre ambas versiones en las seis funcionalidades:

Archivo	Cambio
UpdatePocket.ts	execute dividido en validateInput / loadAuthorized / adjustAmount / notify (cognitiva 16 → 6, ciclomática 17 → 7)
CreatePocket.ts, DeletePocket.ts, TransferPocketBalance.ts, UpdatePocket.ts, DepositMoney.ts	import ... from 'crypto' → 'node:crypto'
pocket.validators.ts	z.string().uuid({ message }) → z.uuid({ error }) (API de zod 4)

El diff completo está en [docs/evidencia-regresion-bolsillos/diff-produccion-antes-despues.patch](#).

7.2 Procedimiento

1. Se creó un *git worktree* aislado en 6102a28.
2. Se copió a ese worktree **el mismo paquete de pruebas**: regresión de las 6 funcionalidades, fakes, helper y suites unitarias de unit/pocket y unit/account.
3. Se ejecutó `vitest run src/tests/regression src/tests/unit/pocket src/tests/unit/account` en ambas versiones, con reporte JSON.
4. Se compararon los resultados **caso por caso** (archivo + nombre completo + estado).

7.3 Resultados

Métrica	Antes (6102a28)	Después (rama del pipeline)
Archivos de prueba	18	18
Casos ejecutados	198	198
Aprobados	193	193
Defectos conocidos (it.fails)	5	5
Fallidos	0	0
Casos con resultado distinto	—	0

Archivo	Casos	Antes	Después
regression/pocket/crear.regression.test.ts	21	✓	✓
regression/pocket/consultar.regression.test.ts	11	✓	✓
regression/pocket/actualizar.regression.test.ts	19	✓	✓

Archivo	Casos	Antes	Después
<code>regression/pocket/eliminar.regression.test.ts</code>	10	✓	✓
<code>regression/pocket/transferir.regression.test.ts</code>	15	✓	✓
<code>regression/pocket/ciclo-de-vida.regression.test.ts</code>	2	✓	✓
<code>regression/account/depositar.regression.test.ts</code>	16	✓	✓
<code>unit/pocket/ — 6 suites con fluent assertions</code>	58	✓	✓
<code>unit/account/DepositMoney.test.ts</code>	12	✓	✓
<code>unit/pocket/ — 4 suites del equipo (controller y repositorio)</code>	34	✓	✓

Test Files 18 passed (18)
Tests 193 passed | 5 expected fail (198)

7.4 Sensibilidad: la suite detecta cambios que rompen el comportamiento

Una suite que pasa antes y después solo sirve si **falla cuando algo se rompe**. Se introdujeron cambios a propósito en el código de producción (y se revirtieron):

Cambio introducido	Resultado de la suite
<code>DeletePocket</code> : restar en vez de sumar el monto al saldo	4 fallan (RG-EL-02, RG-EL-06, RG-FL-01 y 1 unitaria)
<code>TransferPocketBalance</code> : no guardar el bolsillo destino	3 fallan
<code>UpdatePocket</code> : <code>></code> cambiado por <code>>=</code> en la validación de saldo	1 falla (RG-AC-07, valor límite)
<code>DepositMoney</code> : quitar la verificación de propiedad de la cuenta	2 fallan (RG-DE-12 y 1 unitaria)

Al revertir cada cambio, la suite vuelve a **193 aprobadas + 5 it.fails**. (Cambiar `<= 0` por `< 0` en `DepositMoney` **no** se detecta, y es correcto: `!dto.amount` ya rechaza el cero, así que es un cambio *equivalente* que no altera el comportamiento.)

7.5 Conclusión

Las **198 pruebas** producen **exactamente el mismo resultado** antes y después, y la suite **sí detecta** los cambios que alteran el comportamiento. **No hubo regresiones**. Los cinco defectos conocidos siguen presentes en ambas versiones.

8. Integración continua (Jenkins + SonarQube + Docker)

El pipeline del equipo (`Jenkinsfile` en `devops/jenkins-gitops`) ejecuta:

```
Checkout → Backend: install + test + build → Frontend: install + test + build
→ SonarQube analysis → Quality Gate → Docker build → Run with Docker (smoke test)
→ Docker push → Update GitOps repo (FuBanking-gitops) → ArgoCD sincroniza en minikube
```

- La etapa **Backend** corre `npm run test:coverage`. Vitest incluye `src/tests/**/*.test.ts`, así que las 94 pruebas de regresión y las 70 unitarias de este alcance **se ejecutan sin configuración extra**.
- El pipeline genera un `.env` de prueba con `JWT_SECRET`, que es lo único que necesita la suite (el `authMiddleware` real firma y verifica tokens). No usa base de datos ni red.

- `test:coverage` produce `backend/coverage/lcov.info`, que lee el análisis de **SonarQube**.
- Verificado localmente sobre esta rama: **64 archivos, 498 aprobadas + 5 it.fails, 99,6 % de cobertura de líneas y tsc** sin errores.

9. Cobertura

Con `npm run test:regression:bolsillos`, los 9 archivos de las seis funcionalidades quedan en **100 % de líneas, ramas y funciones**: `CreatePocket.ts`, `GetAccountPockets.ts`, `UpdatePocket.ts`, `DeletePocket.ts`, `TransferPocketBalance.ts`, `DepositMoney.ts`, `Pocket.ts`, `PocketController.ts` y `pocket.validators.ts`.

Ejecutando solo la regresión de API, algunas líneas quedan sin cubrir: son **guardas defensivas inalcanzables por la API** (el validador rechaza antes el monto negativo o el cuerpo vacío). Esas las cubren las pruebas unitarias.

10. Cómo ejecutar

```
cd backend
npm run test:regression:bolsillos    # 6 funcionalidades: regresión API + unitarias (198)
npx vitest run src/tests/regression  # solo la regresión de API (94)
npm run test:coverage                # lo mismo que ejecuta Jenkins (suite completa)
```

Requiere el `.env` del backend (`JWT_SECRET`). No requiere Supabase ni red.

11. Archivos entregados

Rutas de pruebas relativas a `backend/src/tests/`.

Archivo	Contenido	Estado
<code>regression/pocket/*.regression.test.ts</code> (6)	Crear, consultar, actualizar, eliminar, transferir y ciclo de vida	Nuevo
<code>regression/account/depositar.regression.test.ts</code>	Funcionalidad 6: depositar	Nuevo
<code>regression/pocket/support/scenario.ts</code>	Escenario base, catálogo de errores y utilidades	Nuevo
<code>helpers/createPocketTestApp.ts</code>	App Express de prueba (Bolsillos + depósito)	Nuevo
<code>fakes/InMemory{Account,Pocket,Notificacion}Repository.ts</code>	Fakes con semántica de copia	Nuevo
<code>unit/account/DepositMoney.test.ts</code>	Unitarias del depósito (AAA + 5 dobles + Chai BDD)	Nuevo
<code>unit/pocket/*.test.ts</code> (6)	Aserciones fluidas Chai BDD	Modificado
<code>unit/pocket/test-doubles.ts</code>	<code>updateStatus</code> en Fake, Stub y Mock	Modificado
<code>backend/package.json</code>	Script <code>test:regression:bolsillos</code>	Modificado
<code>docs/evidencia-regresion-bolsillos/</code>	Logs antes/después, comparación JSON y diff de producción	Nuevo

12. Conclusiones

- Las **seis funcionalidades** quedan protegidas por **94 casos de regresión a nivel de API** que recorren la cadena real de producción y verifican contrato, estado persistido, invariante de dinero, notificaciones y ausencia de efectos parciales.
- Las pruebas usan **aserciones fluidas** (Chai BDD): 70 unitarias y 94 de regresión.
- La comparación **antes vs después** demuestra que los cambios de código no introdujeron regresiones: **198 de 198 casos con el mismo resultado**. Los cambios introducidos a propósito sí se detectan.
- La suite corre dentro del **pipeline de Jenkins**, cuya cobertura alimenta a **SonarQube** antes de construir y ejecutar la aplicación con **Docker**.